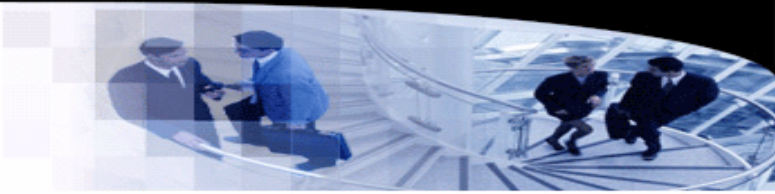
武漢大學
WUHAN UNIVERSITY

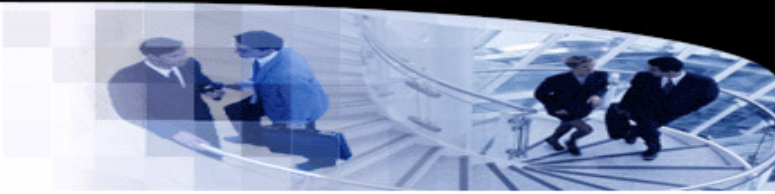


Programming Languages and Compilers

Lecture 26. Global Optimizations

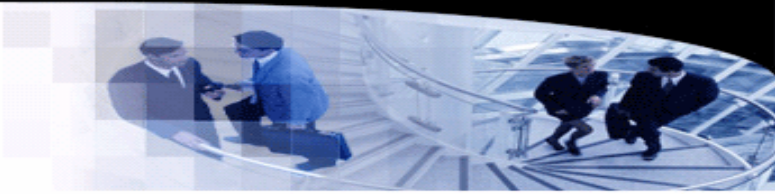
袁梦霆 (YUAN Mengting)
ymt@whu.edu.cn
School of Computer Science, Wuhan University

Global Optimizations



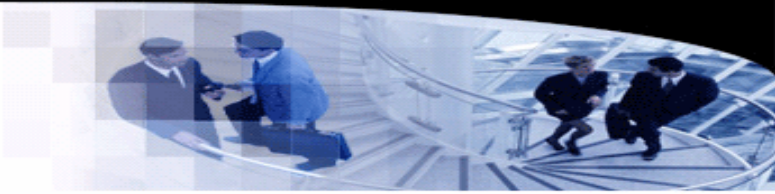
- Introduction
  - Optimizations on CFGs are commonly known as global optimizations.
  - Global optimizations use data-flow analysis to find more optimization opportunities.
- Global optimizations
  - Global constant/copy propagation
  - Global dead code elimination
  - Global common subexpression elimination
  - Code motion
  - Loop optimizations

Global Common Subexpressions



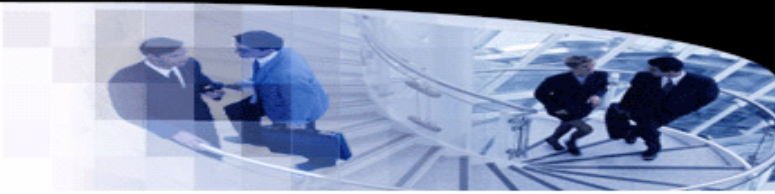
- Available expressions
  - Availability: Has the value of an expression already been computed?
  - Data-flow value of available expressions: a set of expressions that have already been computed.
- Example:
  - `a = b + c;`

Global Common Subexpressions



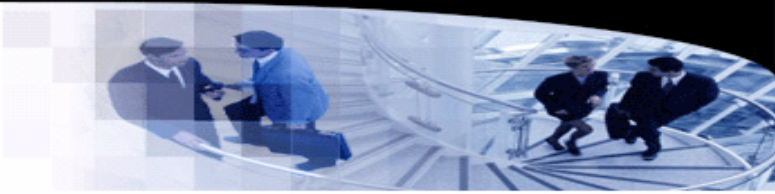
- Available expressions
  - Availability: Has the value of an expression already been computed?
  - Data-flow value of available expressions: a set of expressions that have already been computed.
- Example:
  - `a = b + c;`
  - `{a = b + c} // b + c` has already been computed.

Global Common Subexpressions



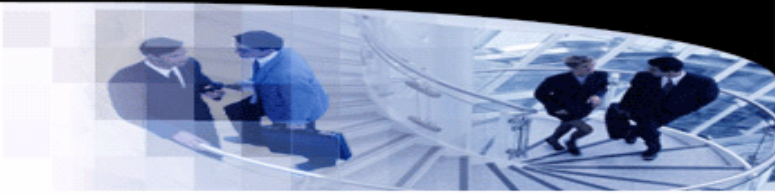
- Available expressions
  - Availability: Has the value of an expression already been computed?
  - Data-flow value of available expressions: a set of expressions that have already been computed.
- Example:
  - `a = b + c;`
  - `{a = b + c} // b + c` has already been computed.
  - `d = b + c;`

Global Common Subexpressions



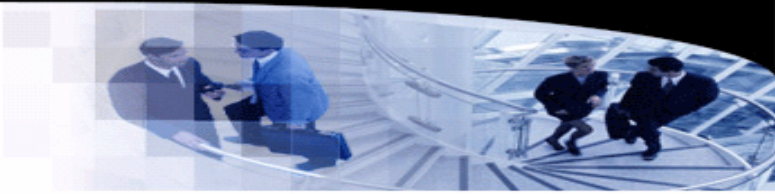
- Available expressions
  - Availability: Has the value of an expression already been computed?
  - Data-flow value of available expressions: a set of expressions that have already been computed.
- Example:
  - `a = b + c;`
  - `{a = b + c}` `// b + c` has already been computed.
  - `d = b + c;` ⟹ `d = a;` `// b + c` is available.
  - `{a = b + c}`

Global Common Subexpressions



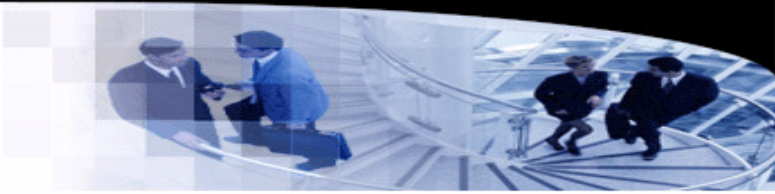
- Available expressions
  - Availability: Has the value of an expression already been computed?
  - Data-flow value of available expressions: a set of expressions that have already been computed.
- Example:
  - `a = b + c;`
  - `{a = b + c}` `// b + c` has already been computed.
  - `d = b + c;` ⟹ `d = a;` `// b + c` is available.
  - `{a = b + c}`
  - `d = e * 2;`
  - `{a = b + c, d = e * 2}`

Global Common Subexpressions



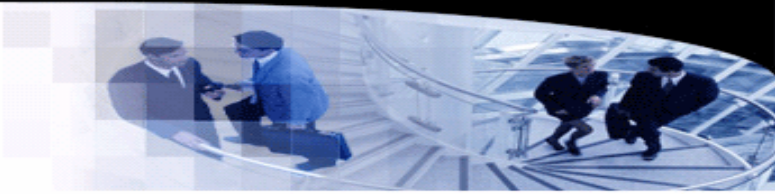
- Available expressions
  - Availability: Has the value of an expression already been computed?
  - Data-flow value of available expressions: a set of expressions that have already been computed.
- Example:
  - `a = b + c;`
  - `{a = b + c}` `// b + c` has already been computed.
  - `d = b + c;` ⟹ `d = a;` `// b + c` is available.
  - `{a = b + c}`
  - `d = e * 2;`
  - `{a = b + c, d = e * 2}`
  - `b = c + 6`
  - ?

Global Common Subexpressions



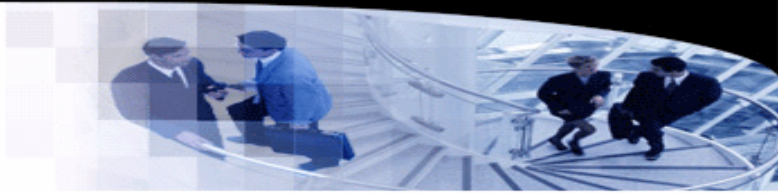
- Available expressions
  - Availability: Has the value of an expression already been computed?
  - Data-flow value of available expressions: a set of expressions that have already been computed.
- Example:
  - `a = b + c;`
  - `{a = b + c}` `// b + c` has already been computed.
  - `d = b + c;` ⟹ `d = a;` `// b + c` is available.
  - `{a = b + c}`
  - `d = e * 2;`
  - `{a = b + c, d = e * 2}`
  - `b = c + 6`
  - `{b = c + 6, d = e * 2}` `// b + c` is not available anymore.

Global Common Subexpressions

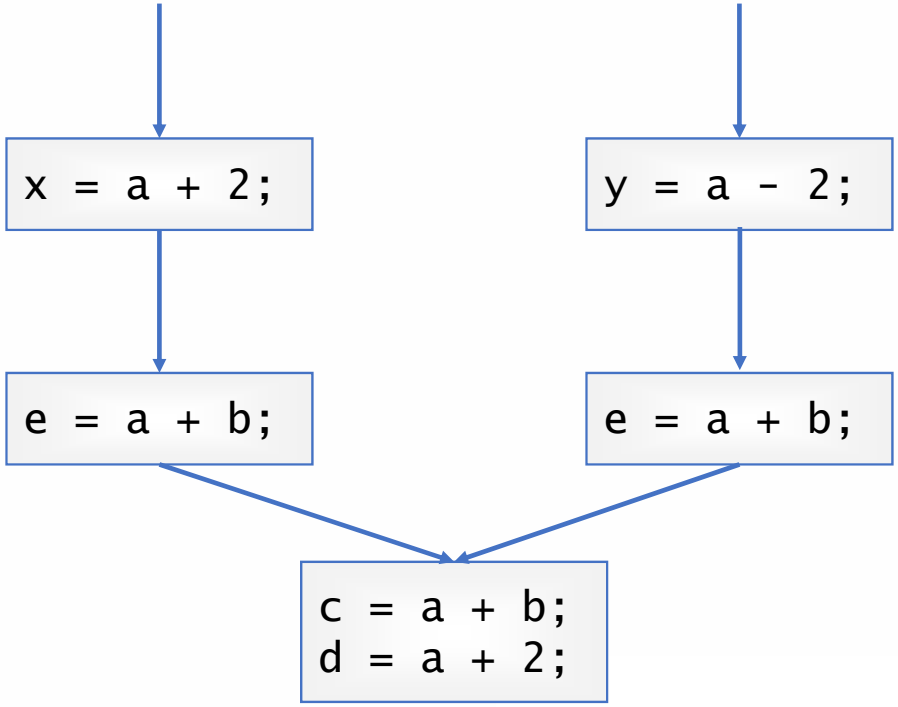


- Available expression analysis
  - Data-flow value of available expressions: a set of expressions that have already been computed.
- Equation over statements
  - Given a statement s: a = b + c, let $IN(s)$ be the set of available expressions before s.
  - $OUT(s)$ is the set of available expressions after s.
  - $gen_s = \{a = b + c\}$ generates a new available expression.
  - $kill_s = \{Exp_a \mid Exp_a \; uses \; a.\}$ kills a set of available expressions.
  - $kill_s$ is the set of **all** expressions that use a.
  - $OUT(s) = (IN(s) \cup gen_s) - kill_s$.
- Example
  - Let s be a = a + b. If $IN(s) = \{c = a + e, d = b + f\}$, how to compute $OUT(s)$?

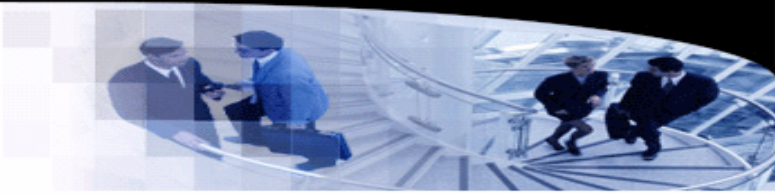
Global Common Subexpressions



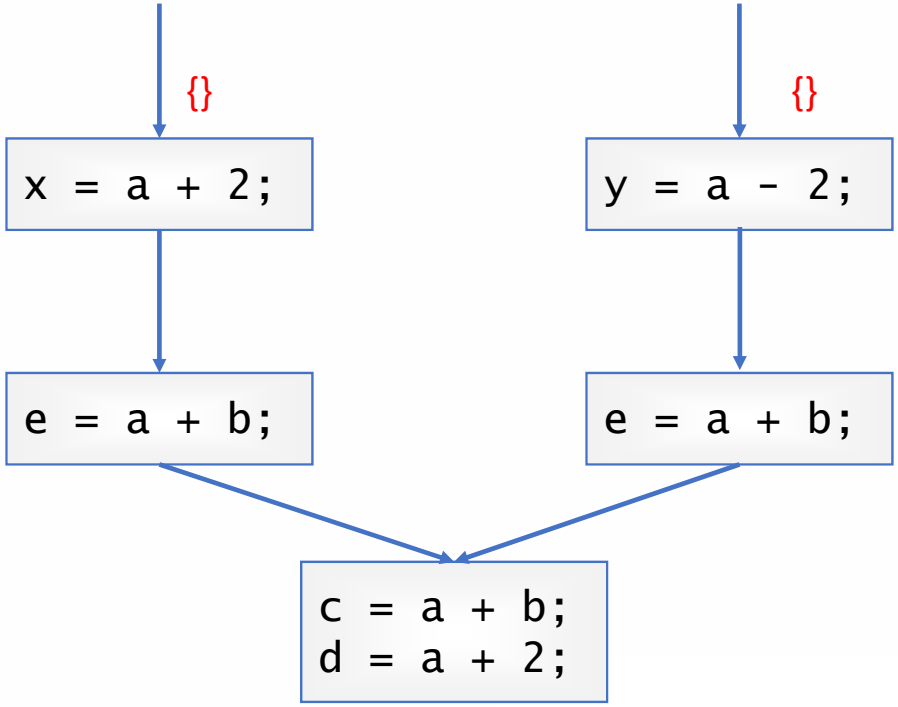
- Equations over basic blocks
  - Given a basic block $B = s_1; s_2; \ldots; s_n$, we can calculate gen_B and $kill_B$ by letting gen_{s_1} go through $s_2; s_3; \ldots; s_n$.
  - $OUT(B) = (IN(B) \cup gen_B) - kill_B$.
- Equations on control flows
  - $IN(B) = \cap_{P \in pred(B)} OUT(P)$.
- Solution
  - The least fixed point.
  - The algorithm starts from $\emptyset$.



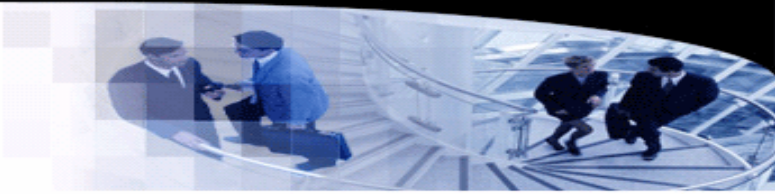
Global Common Subexpressions



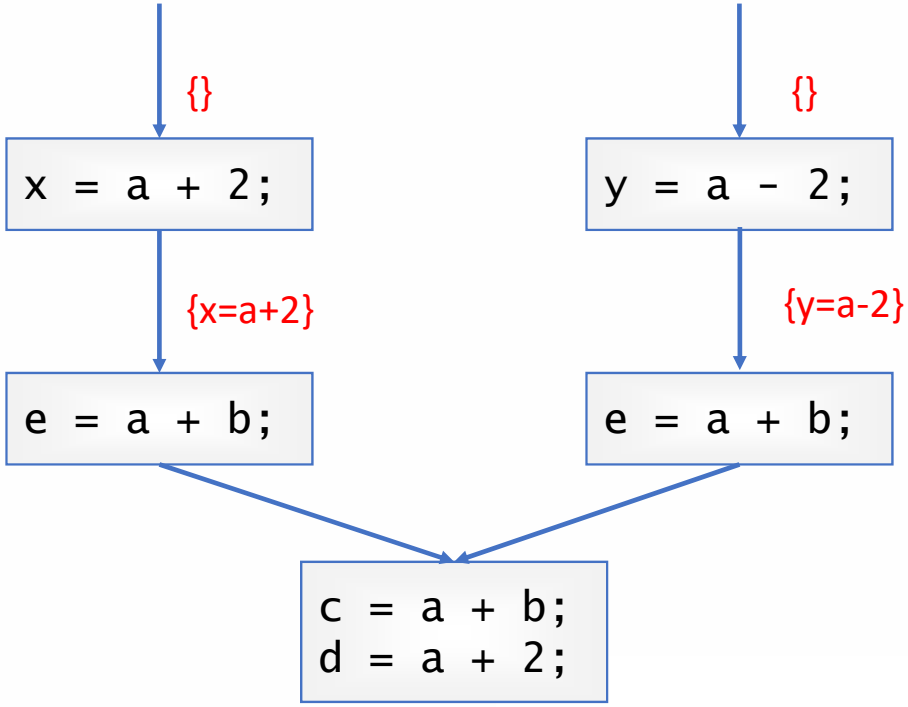
- Equations over basic blocks
  - Given a basic block $B = s_1; s_2; \ldots; s_n$, we can calculate gen_B and $kill_B$ by letting gen_{s_1} go through $s_2; s_3; \ldots; s_n$.
  - $OUT(B) = (IN(B) \cup gen_B) - kill_B$.
- Equations on control flows
  - $IN(B) = \cap_{P \in pred(B)} OUT(P)$.
- Solution
  - The least fixed point.
  - The algorithm starts from $\emptyset$.



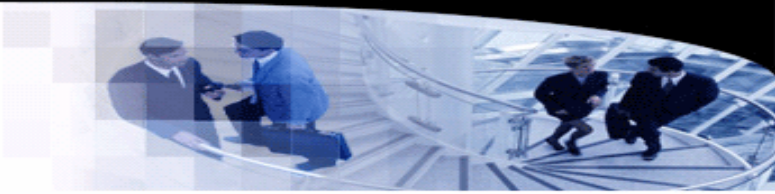
Global Common Subexpressions



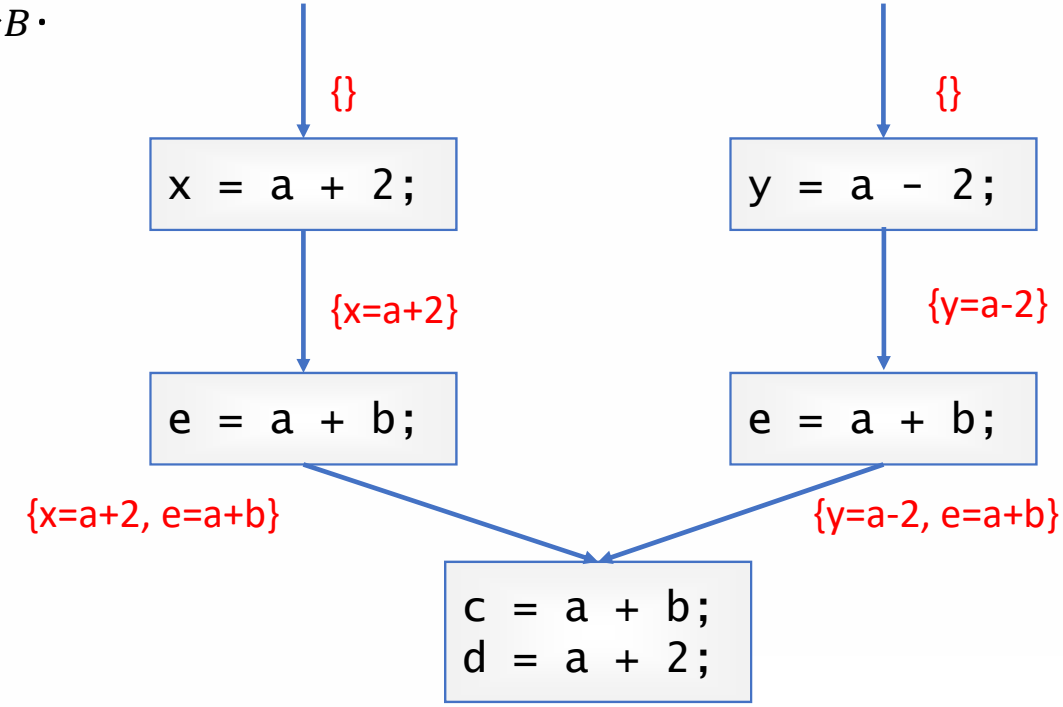
- Equations over basic blocks
  - Given a basic block $B = s_1; s_2; \ldots; s_n$, we can calculate gen_B and $kill_B$ by letting gen_{s_1} go through $s_2; s_3; \ldots; s_n$.
  - $OUT(B) = (IN(B) \cup gen_B) - kill_B$.
- Equations on control flows
  - $IN(B) = \cap_{P \in pred(B)} OUT(P)$.
- Solution
  - The least fixed point.
  - The algorithm starts from $\emptyset$.



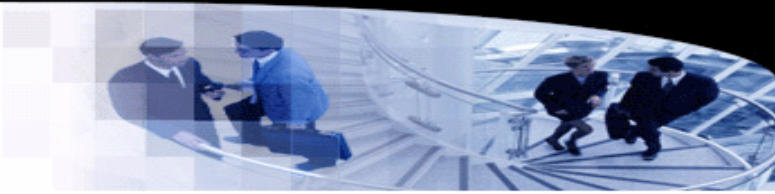
Global Common Subexpressions



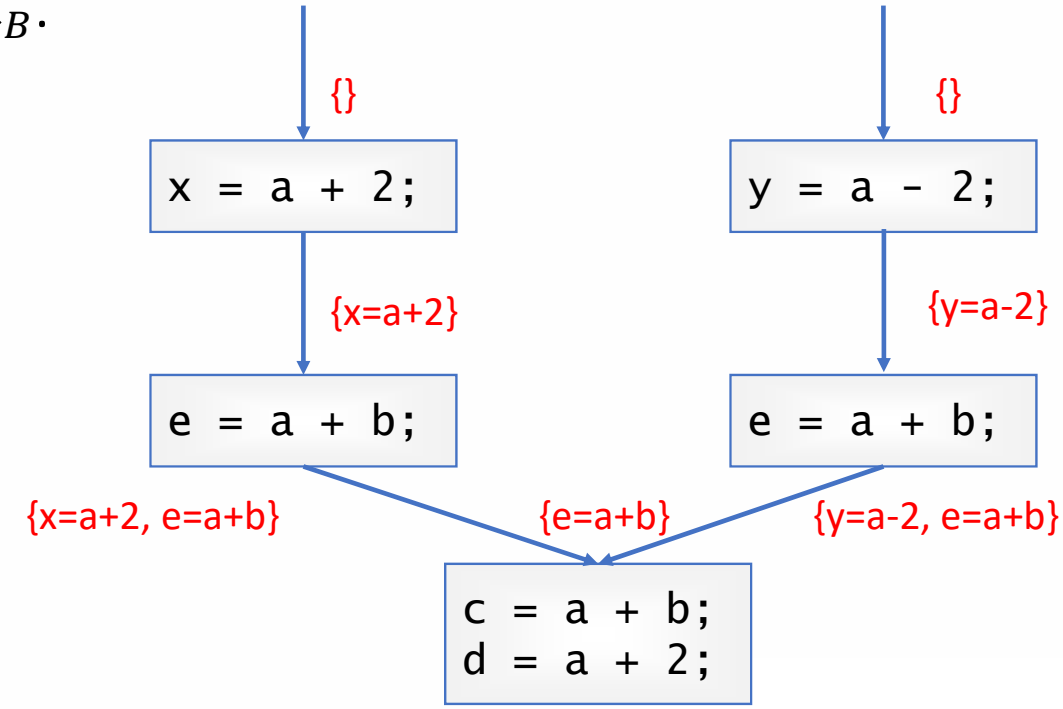
- Equations over basic blocks
  - Given a basic block $B = s_1; s_2; \ldots; s_n$, we can calculate gen_B and $kill_B$ by letting gen_{s_1} go through $s_2; s_3; \ldots; s_n$.
  - $OUT(B) = (IN(B) \cup gen_B) - kill_B$.
- Equations on control flows
  - $IN(B) = \cap_{P \in pred(B)} OUT(P)$.
- Solution
  - The least fixed point.
  - The algorithm starts from ∅.



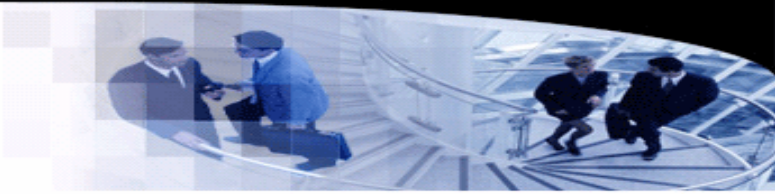
Global Common Subexpressions



- Equations over basic blocks
  - Given a basic block $B = s_1; s_2; \ldots; s_n$, we can calculate gen_B and $kill_B$ by letting gen_{s_1} go through $s_2; s_3; \ldots; s_n$.
  - $OUT(B) = (IN(B) \cup gen_B) - kill_B$.
- Equations on control flows
  - $IN(B) = \cap_{P \in pred(B)} OUT(P)$.
- Solution
  - The least fixed point.
  - The algorithm starts from $\emptyset$.

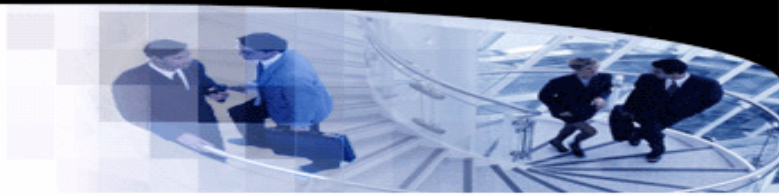


Global Common Subexpressions



- Eliminating common subexpressions
  - With the information of available expressions, it is easy to identify common subexpression.
  - For each statement, we just check whether the right-hand side expression is available.
  - If the expression is available, replace the available expression with the variable (register) that holds the value of the expression.
- Problem:
  - What if there are two common subexpressions that store in different registers?

Loop Optimizations



- Example

```
const int N = 16;

int a[N], b[N];

void foo() {
  for (int i = 0; i < N; ++i)
    a[i] = b[i] + 10;
}
```

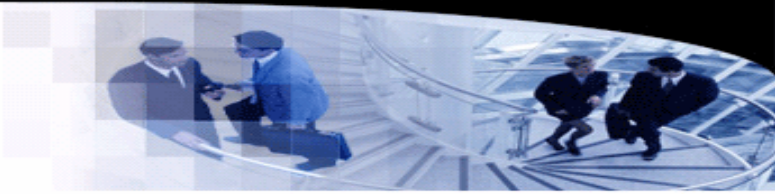
Without optimizations

Optimizations enabled

```
ymt@DESKTOP-VPDNSDJ:~/opt$ clang++ -S -O2 --target=riscv32 -march=rv32imv loop.cpp -o loop.s
ymt@DESKTOP-VPDNSDJ:~/opt$ cat loop.s
        .attribute      4, 16
        .attribute      5, "rv32i2p1_m2p0_f2p2_d2p2_v1p0_zicsr2p0_zmmul1p0_zve32f1p0_zve32x1p0_
x1p0_zvl128b1p0_zvl32b1p0_zvl64b1p0"
        .file   "loop.cpp"
        .text
        .globl  _Z3foov                         # -- Begin function _Z3foov
        .p2align        2
        .type   _Z3foov,@function
_Z3foov:                                # @_Z3foov
# %bb.0:
        lui     a0, %hi(.L_MergedGlobals)
        addi    a0, a0, %lo(.L_MergedGlobals)
        addi    a1, a0, 64
        vsetivli        zero, 8, e32, m2, ta, ma
        vle32.v v8, (a1)
        addi    a1, a0, 96
        vle32.v v10, (a1)
        vadd.vi v8, v8, 10
        vse32.v v8, (a0)
        addi    a0, a0, 32
        vadd.vi v8, v10, 10
        vse32.v v8, (a0)
        ret
```

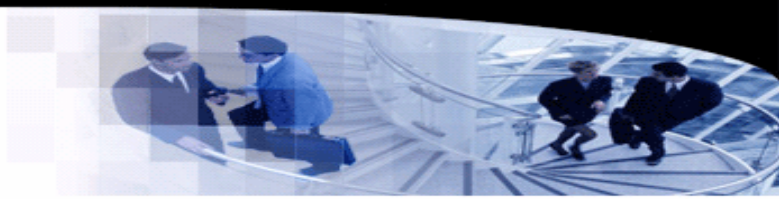
```
ymt@DESKTOP-VPDNSDJ:~/opt$ clang++ -S -O0 --target=riscv32 -march=rv32imv loop.cpp -o loop.s
ymt@DESKTOP-VPDNSDJ:~/opt$ cat loop.s
        .attribute      4, 16
        .attribute      5, "rv32i2p1_m2p0_f2p2_d2p2_v1p0_zicsr2p0_zmmul1p0_zve32f1p0_zve32x1p
4b1p0"
        .file   "loop.cpp"
        .text
        .globl  _Z3foov                         # -- Begin function _Z3foov
        .p2align        2
        .type   _Z3foov,@function
_Z3foov:                                # @_Z3foov
# %bb.0:
        addi    sp, sp, -16
        sw      ra, 12(sp)                      # 4-byte Folded Spill
        sw      s0, 8(sp)                       # 4-byte Folded Spill
        addi    s0, sp, 16
        li      a0, 0
        sw      a0, -12(s0)
        j       .LBB0_1
.LBB0_1:                                # =>This Inner Loop Header: Depth=1
        lw      a1, -12(s0)
        li      a0, 15
        blt     a0, a1, .LBB0_4
        j       .LBB0_2
.LBB0_2:                                #   in Loop: Header=BB0_1 Depth=1
        lw      a0, -12(s0)
        slli    a2, a0, 2
        lui     a0, %hi(b)
        addi    a0, a0, %lo(b)
        add     a0, a0, a2
        lw      a0, 0(a0)
        addi    a0, a0, 10
        lui     a1, %hi(a)
        addi    a1, a1, %lo(a)
        add     a1, a1, a2
        sw      a0, 0(a1)
        j       .LBB0_3
.LBB0_3:                                #   in Loop: Header=BB0_1 Depth=1
        lw      a0, -12(s0)
        addi    a0, a0, 1
        sw      a0, -12(s0)
        j       .LBB0_1
.LBB0_4:
        addi    sp, s0, -16
        lw      ra, 12(sp)                      # 4-byte Folded Reload
```

Loop Optimizations



- Introduction
  - Loop optimizations are critical to performance, as programs typically spend most of their running time in loops.
  - Given a program represented as a CFG, a loop is a set of strongly connected basic blocks.
- Common loop optimizations
  - Loop unrolling
  - Loop fusion and loop fission
  - Invariant removal
  - Loop tiling
  - Induction variable elimination

Loop Optimizations



- Loop unrolling
  - Unwind loops by eliminating loop control instructions.
  - Use space to trade speed.

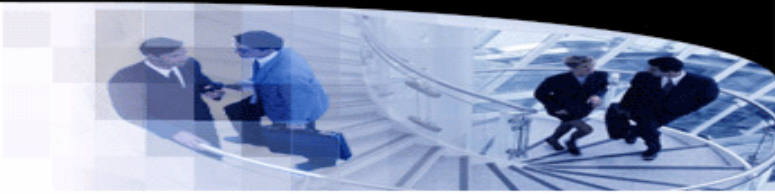
```
const int N = 4;

int a[N], b[N];

void foo() {
  for (int i = 0; i < N; ++i)
    a[i] = b[i] + 1;
}
```

```
ymt@DESKTOP-VPDNSDJ:~/loop$ clang++ -S -O2 --target=riscv32 unroll.cpp -o unroll.s
ymt@DESKTOP-VPDNSDJ:~/loop$ cat unroll.s
        .attribute      4, 16
        .attribute      5, "rv32i2p1_m2p0_a2p1_c2p0_zmmul1p0_zaamo1p0_zalrsc1p0"
        .file   "unroll.cpp"
        .text
        .globl  _Z3foov                         # -- Begin function _Z3foov
        .p2align        1
        .type   _Z3foov,@function
_Z3foov:                                        # @_Z3foov
# %bb.0:
        lui     a0, %hi(.L_MergedGlobals)
        addi    a1, a0, %lo(.L_MergedGlobals)
        lw      a2, 16(a1)
        lw      a3, 20(a1)
        lw      a4, 24(a1)
        lw      a5, 28(a1)
        addi    a2, a2, 1
        sw      a2, %lo(.L_MergedGlobals)(a0)
        addi    a3, a3, 1
        addi    a4, a4, 1
        addi    a5, a5, 1
        sw      a3, 4(a1)
        sw      a4, 8(a1)
        sw      a5, 12(a1)
        ret
```

Loop Optimizations



- Loop fusion
  - Transform multiple loops into a single one.
  - Reduce control-flow instructions.
  - Improve parallelism.

```
for (int i = 0; i < N; ++i)
  a[i] = i;
for (int i = 0; i < N; ++i)
  b[i] = i * 2;
```

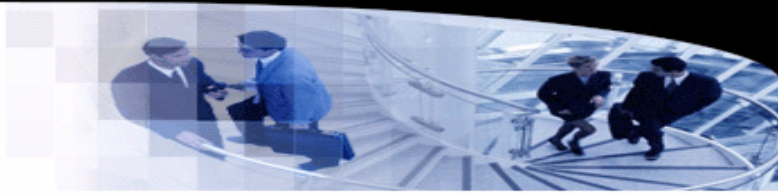
Fusion

Fission

```
for (int i0 = 0, i1 = 0;
     i0 < N, i1 < N;
     ++i0, ++i1) {
  a[i0] = i0;
  b[i1] = i1 * 2;
}
```

- Loop fission
  - Break a large loop into small ones.
  - Better data locality.
  - Provide benefits to multi-core processors.

Loop Optimizations



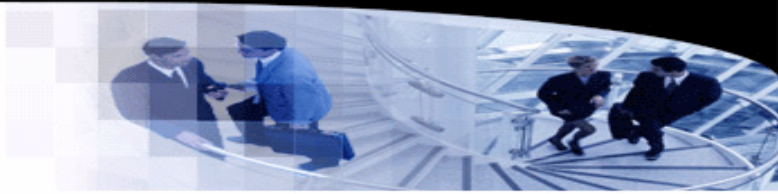
- Loop tiling
  - Change the iteration order to improve cache performance for nested loops.
  - Example: matrix-vector multiplication.

```
for (int i = 0; i < N; ++i) {
   c[i] = 0;
   for (int j = 0; j < N; ++j)
     c[i] = c[i] + a[i][j] * b[j];
 }
```

Tiling

```
for (int i = 0; i < N; i += 2) {
  c[i] = 0;
  c[i + 1] = 0;
  for (int j = 0; j < N; j += 2) {
    for (int x = i; x < min(i + 2, N); ++x) {
      for (int y = j; y < min(j + 2, N); ++y) {
        c[x] = c[x] + a[x][y] * b[y];
      }
    }
  }
}
```

Loop Optimizations



- Loop invariant removal
  - Consider a statement `s: a = b + c`.
  - `s` is invariant for a loop, if for each used value `v` either
    - `v` is a constant, or
    - all the reaching definitions of `v` are outside the loop, or
    - there is only one reaching definition of `v`, provided it is a loop invariant.
  - In this case, `s` can be moved out of the loop.

```
for (int i = 0; i < N; ++i) {
  c[i] = 0;
  x = a + 4;
}
```

Removal

```
x = a + 4;
for (int i = 0; i < N; ++i) {
  c[i] = 0;
}
```

  - Are these removals always safe?